Initial Design Plan

Ronit Mehta

October 21, 2026

This document lays out the design plan for this project, relating to the tasks that each member will be doing and how each part of the project will be structured.

This is just the first draft and many things will probably change about the design as we start working on the project and discussing the problems we are running into. We can discuss any changes we might want to make to the design anytime.

Naming Conventions

For the sake of readability and ease of use, please follow the snake_case naming convention for variables and functions(where all lowercase letters are used and different words are seperated by an underscore) and the PascalCase naming convention for classes(where there is no space between different words and the start of each word is uppercase).

Try not to give ambiguous names to variables, functions and classes as it makes reading the code hard. Make sure that the name of a variable, function or class clearly communicates its purpose. In a situation where this is hard to do, use comments to be extra clear.

These are not hard rules just some guidelines to keep in mind.

Project layout

There will be three source files:

1. algorithms.py(Ronit and Phani):
   Optimization Class with attributes objective, gradient and hessian. Methods to be included are steepest_descent() and newton_method(), each taking the necessary arguments for the method. Each of the methods should return final vector, iteration history and iteration count, in that order.

2. tests.py(Sayani):
   For each test function required(As mentioned in the doc given by sir), write a seperate class and put all the relevant information about the testing of that function in that class and do so with the same naming convention for each of the classes so that regardless of which test class is being accessed, they all have the same internal structure. The following should be included:
   (a) The objective function definition as a static method
   (b) The gradient definition as a static method
   (c) The hessian definition as a static method
   (d) The analytic solution to test against for convergence
   (e) The stopping criterion
3. benchmarks.py(Swapnil):
   Use the classes mentioned above to run and generate the necessary benchmarks and visuals for the different test functions. To avoid repetition, this can be done with a Benchmark Class which takes an unkown class(with the format of a test class mentioned above) during initialization and extracts the necessary information from it.

Throughout the project we will be using numpy and will try to use numpy float arrays for vectors and matrices. We can change this later to use python arrays, but there is no need for that now.

For the github collaboration workflow refer to the doc given by sir.

Things not mentioned here that can be added later:

1. Debug modes
2. Automating tests
3. Random tests
4. Implementation of the algorithms without being given the gradient and hessian of the objective functions